

PRACTICAL WEEK-11

NAME:CH.SAI VIHAMSH

ROLL:2520090090

SESSION-11

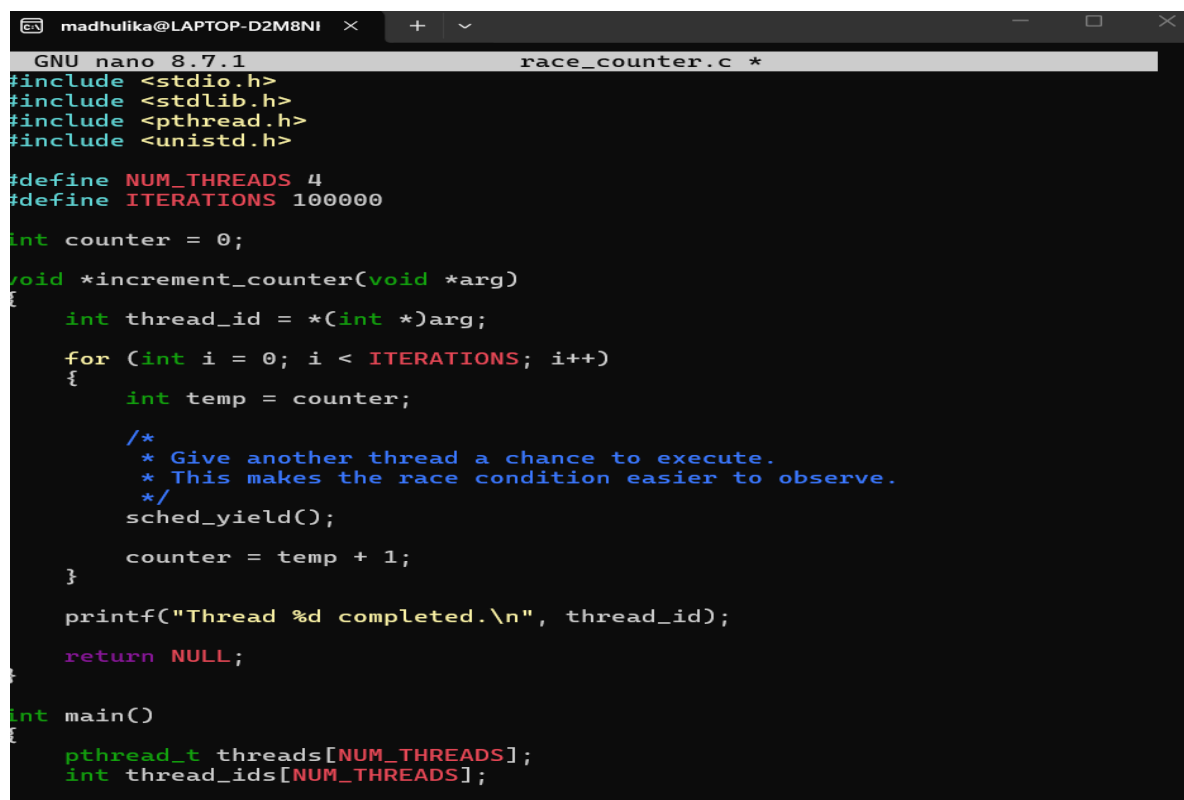
Task1: Develop a multithreaded counter application using POSIX threads (pthread_create, pthread_join). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.

```
saivihamsh@Varshini-PC:~$ cd ~/OSSP/practical
saivihamsh@Varshini-PC:~/OSSP/practical$ mkdir -p week11/task1
saivihamsh@Varshini-PC:~/OSSP/practical$ cd week11/task1
saivihamsh@Varshini-PC:~/OSSP/practical/week11/task1$ nano race_counter.c
-bash: cd: week11/task1: No such file or directory
saivihamsh@Varshini-PC:~/OSSP/practical/week11/task1$ nano race_counter.c
saivihamsh@Varshini-PC:~/OSSP/practical/week11/task1$ gcc race_counter.c -o T1
total 20
saivihamsh@Varshini-PC:~/OSSP/practical/week11/task1$ ls -l
total 20
-rwxr-xr-x 1 madhulika madhulika 16312 Sep 19 03:07 T1
-rw-r--r-- 1 madhulika madhulika 2018 Sep 19 03:07 race_counter.c
saivihamsh@Varshini-PC:~/OSSP/practical/week11/task1$ ./T1
=== Practical Week 11 Task 1 ===
POSIX Threads and Race Condition

Number of threads : 4
Iterations/thread : 100000
Expected counter : 400000

Thread 2 completed.
Thread 3 completed.
Thread 1 completed.
Thread 4 completed.

All threads completed.
Actual counter : 104700
Race condition detected.
Some increments were lost because threads accessed the shared variable concurrently.
saivihamsh@Varshini-PC:~/OSSP/practical/week11/task1$
```



```
GNU nano 8.7.1 race_counter.c *
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define NUM_THREADS 4
#define ITERATIONS 100000

int counter = 0;

void *increment_counter(void *arg)
{
    int thread_id = *(int *)arg;

    for (int i = 0; i < ITERATIONS; i++)
    {
        int temp = counter;

        /*
         * Give another thread a chance to execute.
         * This makes the race condition easier to observe.
         */
        sched_yield();

        counter = temp + 1;
    }

    printf("Thread %d completed.\n", thread_id);

    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];
    int thread_ids[NUM_THREADS];
```

```
GNU nano 8.7.1 race_counter.c *
{
    perror("pthread_create");
    return 1;
}

/*
 * Wait for all threads
 */
for (int i = 0; i < NUM_THREADS; i++)
{
    if (pthread_join(threads[i], NULL) != 0)
    {
        perror("pthread_join");
        return 1;
    }
}

printf("\nAll threads completed.\n");
printf("Actual counter      : %d\n", counter);

if (counter != NUM_THREADS * ITERATIONS)
{
    printf("Race condition detected.\n");
    printf("Some increments were lost because "
           "threads accessed the shared variable concurrently.\n");
}
else
{
    printf("Race condition was not visible in this run.\n");
    printf("Run the program again to observe the effect.\n");
}

return 0;
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

Observation: The program successfully created multiple POSIX threads using `pthread_create()` and waited for their completion using `pthread_join()`. All threads accessed and modified the same shared counter variable concurrently without using any synchronization mechanism. Since the increment operation involves reading, modifying, and writing the shared variable, multiple threads could access the counter at the same time and overwrite each other's updates. As a result, the actual counter value was generally different from the expected value of 400000, demonstrating the effect of a race condition in a multithreaded program.

Task2: Modify the previous program using mutex locks (pthread_mutex_lock, pthread_mutex_unlock) and compare the outputs with the race-condition version. Analyze the impact of synchronization.

```
saiviamsh@Varshini-PC:~/OSSP/practical/week11/task2$ cd ~/OSSP/practical
saiviamsh@Varshini-PC:~/OSSP/practical$ mkdir -p week11/task2
saiviamsh@Varshini-PC:~/OSSP/practical$ cd week11/task2
saiviamsh@Varshini-PC:~/OSSP/practical/week11/task2$ nano mutex_counter.c
saiviamsh@Varshini-PC:~/OSSP/practical/week11/task2$ gcc mutex_counter.c -o T2 -pthread
saiviamsh@Varshini-PC:~/OSSP/practical/week11/task2$ ./T2
=== Practical Week 11 - Task 2 ===
POSIX Threads with Mutex Synchronization

Number of threads : 4
Iterations/thread : 100000
Expected counter : 400000

Thread 4 completed.
Thread 1 completed.
Thread 3 completed.
Thread 2 completed.

All threads completed.
Actual counter : 400000
Synchronization successful.
No increments were lost.
Mutex destroyed successfully.
saiviamsh@Varshini-PC:~/OSSP/practical/week11/task2$
```

```
GNU nano 8.7.1 mutex_counter.c *
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NUM_THREADS 4
#define ITERATIONS 100000

int counter = 0;
pthread_mutex_t lock;

void *increment_counter(void *arg)
{
    int thread_id = *(int *)arg;
    for (int i = 0; i < ITERATIONS; i++)
    {
        /*
         * Lock before accessing the shared variable.
         */
        pthread_mutex_lock(&lock);

        counter++;

        /*
         * Unlock after updating the shared variable.
         */
        pthread_mutex_unlock(&lock);
    }
    printf("Thread %d completed.\n", thread_id);
    return NULL;
}

int main()
{
```

```
GNU nano 8.7.1 mutex_counter.c *
/*
 * Wait for all threads
 */
for (int i = 0; i < NUM_THREADS; i++)
{
    if (pthread_join(threads[i], NULL) != 0)
    {
        perror("pthread_join");
        pthread_mutex_destroy(&lock);
        return 1;
    }
}

printf("\nAll threads completed.\n");

printf("Actual counter    : %d\n", counter);

if (counter == NUM_THREADS * ITERATIONS)
{
    printf("Synchronization successful.\n");
    printf("No increments were lost.\n");
}
else
{
    printf("Unexpected counter value.\n");
}

/*
 * Destroy mutex
 */
pthread_mutex_destroy(&lock);

printf("Mutex destroyed successfully.\n");

return 0;
}
```

Observation: The race-condition program was modified by introducing a POSIX mutex to synchronize access to the shared counter. Before modifying the counter, each thread acquired the mutex using `pthread_mutex_lock()`, and after completing the update it released the mutex using `pthread_mutex_unlock()`. This ensured that only one thread could modify the shared counter at a time. The program was executed multiple times and consistently produced the expected counter value of 400000, showing that synchronization prevented lost updates caused by concurrent access.